Variable Functions

So far we've called functions by using their names as symbols in our code, but in fact any expression, which evaluates to a string can also be used provided the string matches the function name. This is demonstrated below:

```
$funcName = 'myFunc';
$funcName();

function myFunc (): void
{
    print "Hello from myFunc" . PHP_EOL;
}

Output:
Hello from myFunc
```

Hopefully this should make immediate sense. Rather than write `myFunc();` we have instead used an expression which will evaluate to the string literal value `"myFunc"`. If parentheses are placed immediately after the expression, the corresponding function is called. The variable `$funcNanme` contains the string `"myFunc"`, so when the PHP engine encounters the expression `$funcName();` at run-time (reading from right to left), first the parentheses are recognized, then the variable name. The parentheses indicate a function call, but first the variable needs to be evaluated. PHP retrieves the string from memory, checks if `myFunc()` is declared, and calls it as normal. If the variable does not match a known function, then a fatal "Call to undefined function" error is the result.

Incidentally, I really do mean "any expression." Let's have a little fun:

```
'myFunc'();
('my' . 'Func')();

$my = 'my';
$func = 'Func';
($my . $func)();

function myFunc (): void
{
    print "Let's, ahem, get func-y" . PHP_EOL;
}
```